

Let's completely clear the slate and lock this down from square one. You have forked the **rebelytics/one-skill-to-rule-them-all** repository. [1]

Here is exactly how to link your fork, your two MCP layers, and your Honey harness directly into the codebase memory loop so Claude Code actually follows it. [2]

Step 1: Install Your task-observer Fork Scoped to Your Repo

Since you want this isolated to your current setup, place your forked folder inside your repository's local project-level skill tree. [2]

1. Create the project skills path inside your active workspace directory:

```
mkdir -p .claude/skills/
```

2. Move or clone your personal fork of the one-skill-to-rule-them-all repository straight into that folder, ensuring the directory is named task-observer:

```
# Make sure the inner subfolders like /references and /scripts stay intact
```

```
mv /path/to/your-forked-repo .claude/skills/task-observer
```

[2]

Step 2: Inject Your Three Layers into the Meta-Skill Memory Base

The task-observer works by tracking your execution patterns and checking them against a global reference. We need to feed it a file called cross-cutting-principles.md inside its working track path so it knows how to handle your specific 3-layer stack. [3, 4]

1. Create the observation log hierarchy at your repo root:

```
mkdir -p skill-observations/
```

2. Create the principles rule file at skill-observations/cross-cutting-principles.md:

```
# CROSS-CUTTING COMPREHENSIVE PRINCIPLES
```

```
- **Layer 1 (Static Corpus):** You MUST query the local  
`terrestrial-brain` MCP server before restructuring files or  
reading directory trees.
```

```
- **Layer 2 (Hot Session Context):** Commit incremental state  
updates and session decisions directly to the  
`mem0-session-memory` MCP server command-by-command.
```

```
- **Layer 3 (Harness Constraints):** Apply Honey for Devs ESON  
optimization rules—strip out all chat padding and conversational  
boilerplate text.
```

[2, 4]

Step 3: Hard-Code the Session-Start Trigger into CLAUDE.md

Description matching alone will under-trigger a meta-skill in Claude Code. To force Claude to activate the task-observer on its absolute *first* turn before it writes code or loops tools, you must write an explicit activation instruction into your root CLAUDE.md. [2, 5]

Create or completely replace the CLAUDE.md file at the root of your project with this exact structural directive: [2]

```
# WORKSPACE METRIC MANAGEMENT
```

```
## Core Meta-Skill Activation
```

```
IMPORTANT: At the start of any task-oriented session – any  
interaction where you will use tools and produce deliverables – you  
MUST invoke the task-observer skill before beginning work or  
writing a plan.
```

```
## Background Architecture Configuration
```

```
- **The Core Harness:** `task-observer` tracks your execution  
performance. Always align operations with the rules in  
`skill-observations/cross-cutting-principles.md`.  
- **The Data Target:** Your attached Obsidian vault share folder  
and source code corpus share the same path. Read notes as  
architecture definitions, not production scripts.
```

Verifying It Works

Close your current terminal window, open a new session, navigate to your repo, and boot the TUI: [2]

```
cd /path/to/your-combined-code-and-vault-repo  
claude
```

Give it any simple command. If configured correctly, **Claude will invoke the task-observer meta-skill before it attempts its first tool check or file scan.** Over the course of a few coding sessions, you will see a local skill-observations/observation-log/ folder automatically generate on your machine, proving the execution loop is live. [2, 5]

- [1] <https://github.com>
- [2] <https://github.com>
- [3] <https://www.claudecodehq.com>
- [4] <https://www.claudecodehq.com>
- [5] <https://github.com>